

Functional Requirements & System Architecture

What the platform will do, and how it will be built.

Document 1 explained why this platform needs to exist. This report picks up from there: it defines the exact features, user roles, and workflows the product must support (Document 2 — FRD), then translates that into a concrete backend architecture, tech stack, and data flow (Document 3 — SAD/HLD).

A Quick Note Before You Dive In

Two documents, one continuous thread — from "what" to "how."

Document 2 (Functional Requirements) defines **what** gets built: the user roles, permissions, modules, and workflows. Document 3 (System Architecture) defines **how** it gets built: the services, tech stack, data flow, and deployment model. Read together, they hand a complete, unambiguous brief to an engineering team.

■

CURIOUS?

Every feature in Document 2 depends on *who* is using it — that's why user roles and permissions come first, before any feature list. It's the same reason Document 3 starts with architecture style, not code.

What's Inside

\$	Document 2 — Functional Requirements
01	Product Overview & Purpose
02	Who Uses the Platform
03	Role & Permission Matrix
04	Core Functional Modules
05	Product Flow
06	Functional Goals & Business Constraints
07	MVP Scope — In vs. Out
08	Designing Around Business Domains

\$	Document 3 — System Architecture & Technical Design
09	System Overview & Architecture Style
10	High-Level Architecture
11	Recommended Tech Stack
12	Core Domains & Backend Modules
13	Request Flow — Creating a Rule
14	Runtime Price Flow
15	Core Pricing Pipeline
16	Non-Functional Requirements
17	Deployment Architecture (MVP)
18	Principal Engineer Review — Three Core Engines
19	The Real Reframe — A Rules Engine, Not a Dashboard

Product Overview

What the system is, in one page.

Attribute	Detail
Product Name	Dynamic Pricing & Revenue Management Platform
Product Type	Internal Enterprise Web Application
Primary Purpose	A centralized platform to configure, simulate, approve, and publish pricing rules across every booking channel.

Phase 1 Deliverables

By the end of this phase, the following are clearly defined:

- Product purpose
- Target users
- User permissions
- Core modules
- Product workflow
- MVP scope

WHY ROLES COME FIRST

A good FRD isn't written feature-by-feature. It starts with **who uses the system**, because every feature — every button, every screen — depends on who's allowed to touch it. That's the order this document follows.

Who Uses the Platform

Six roles, each with a distinct job to do.

Role	Responsibility
Super Admin	Complete system management
Revenue Manager	Owns pricing strategy & rules
Pricing Analyst	Creates and simulates rules
Operations Team	Publishes & monitors live pricing
Finance Team	Audits & reports on outcomes
Business Admin	Runs promotions & campaigns

Role & Permission Matrix

Module	Super Admin	Revenue Mgr	Analyst	Operations	Finance
Manage Users	✓	–	–	–	–
Create Rules	✓	✓	✓	–	–
Edit Rules	✓	✓	✓	–	–
Publish Rules	✓	✓	–	✓	–
Delete Rules	✓	✓	–	–	–
Promotions	✓	✓	✓	✓	–
Analytics	✓	✓	✓	✓	✓
Audit Logs	✓	✓	–	–	✓

■ CURIOUS?

Only two roles can publish a live price change — Revenue Manager and Operations. That narrow gate is deliberate: it keeps a bad rule from ever reaching a real customer without a second set of eyes.

Core Functional Modules

Eleven modules, each owning one job.

Dashboard Business overview at a glance	Pricing Rules Create & manage rules	Rule Engine Executes pricing logic
Price Simulator Test pricing before publish	Promotions Manage campaigns	Channel Pricing Platform-specific pricing
Customer Segments Customer-based pricing	Calendar Seasons & holidays	Reports Revenue analytics
Audit Logs Tracks every change	Settings System configuration	

Product Flow

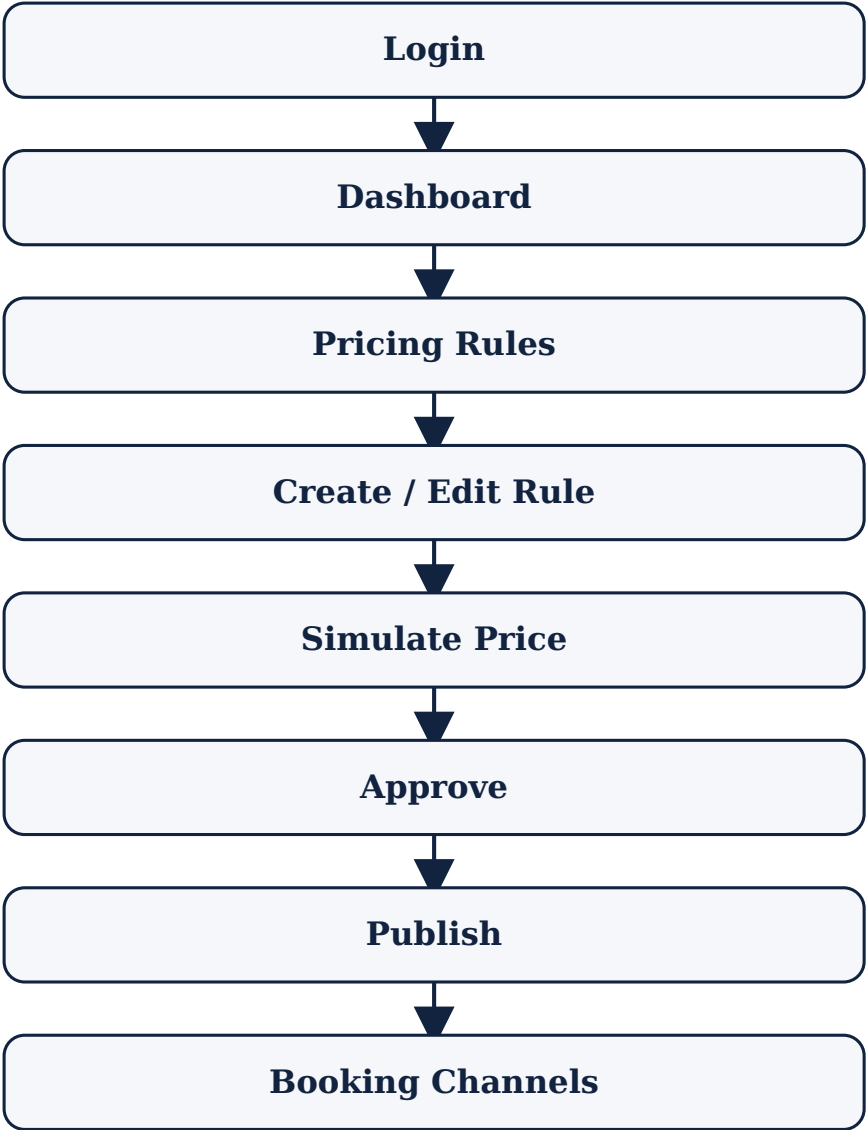


Figure A — The path a business user follows from login to a live, publishable price.

Functional Goals & Business Constraints

What users must be able to do, and what the system must always support.

The Platform Should Let Users

- Create pricing rules
- Modify pricing rules
- Schedule pricing rules
- Simulate pricing
- Publish pricing
- Rollback pricing
- Manage promotions
- Configure channels
- View reports
- Monitor rule execution

The System Must Always Support

- Multi-company
- Multi-cruise
- Multi-route
- Multi-currency
- Multi-channel
- Multi-user
- Role-based access
- Rule versioning

MVP Scope

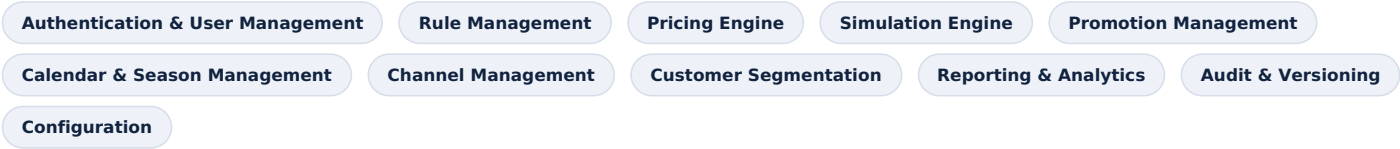
✓ INCLUDED IN MVP - Authentication - Dashboard - Rule Management - Rule Engine - Price Simulator - Channel Pricing - Promotions - Audit Logs - Reports - API for Price Calculation	× FUTURE SCOPE - AI Pricing - Competitor Monitoring - Demand Forecasting - Machine Learning - Auto Price Recommendations
---	---

Designing Around Business Domains

Not a simple CRUD app — a set of connected business domains.

⚠ **STAFF ENGINEER OBSERVATION**

After seeing the complete picture, this shouldn't be designed as a simple CRUD application. Dividing the system into business domains — instead of generic screens — will directly shape the backend architecture: its modules, services, database, and APIs. This keeps the product modular and gives Document 3 a much stronger foundation to build on.



Each domain owns its own APIs, database tables, business logic, validation, and events — a boundary that carries straight through into Document 3's module structure.

System Overview & Architecture Style

A single source of truth for every pricing decision.

The objective is a scalable Dynamic Pricing & Revenue Management Platform that lets business users configure pricing rules, simulate outcomes, approve changes, and expose calculated prices through APIs to every booking system. The platform is the single source of truth for pricing.

Architecture Style

Recommended	Modular Monolith (MVP) — not Microservices
Faster development	One codebase, one deploy pipeline, fewer moving parts to coordinate
Easier deployment	A single service to ship and roll back
Lower infrastructure cost	No inter-service network, no service mesh overhead
Simpler debugging & testing	One process, one set of logs
Better for startup-sized teams	Microservices are introduced later, only when scale demands them

High-Level Architecture

One dashboard, one gateway, six domain services, one source of truth.

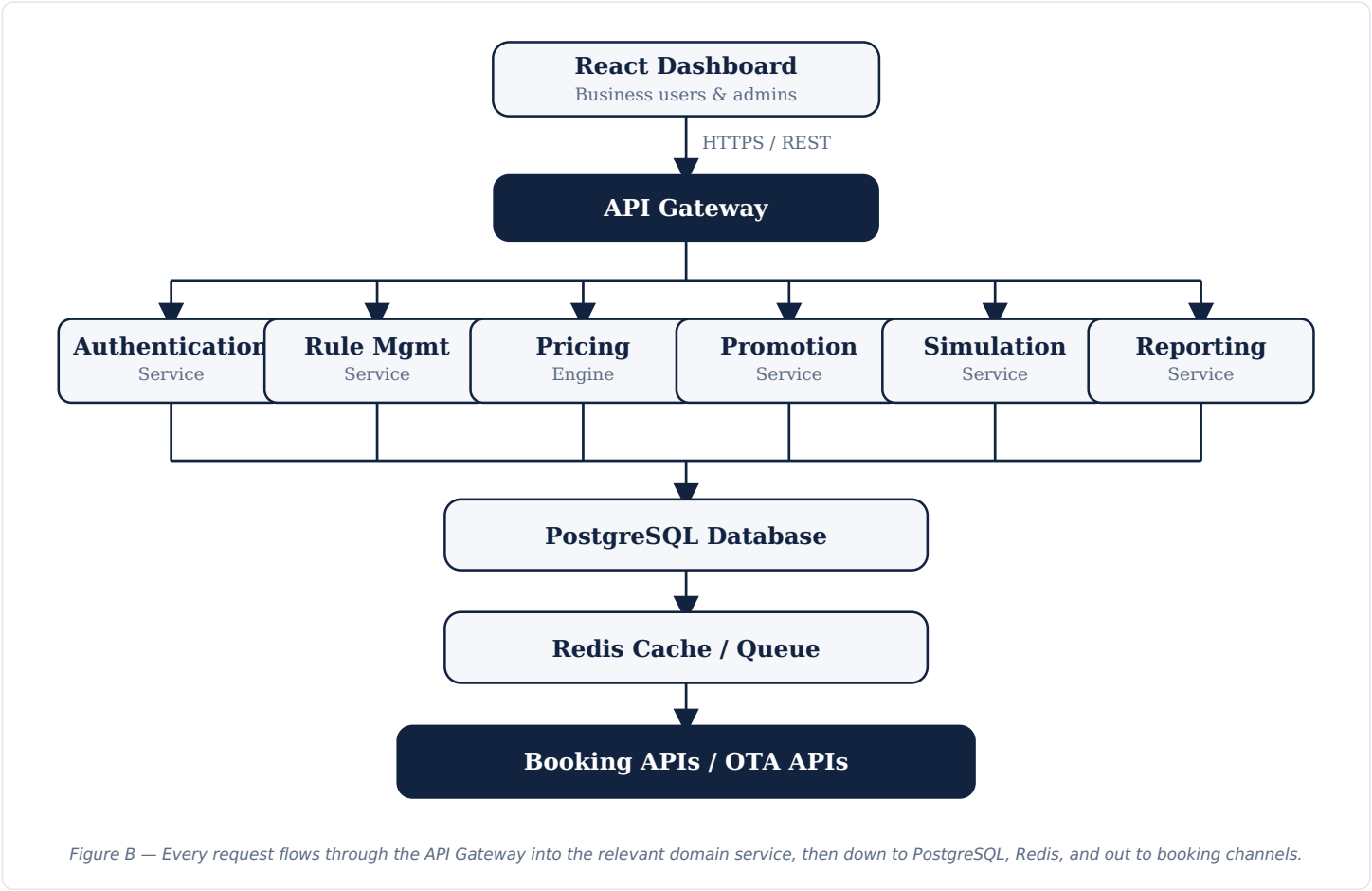


Figure B — Every request flows through the API Gateway into the relevant domain service, then down to PostgreSQL, Redis, and out to booking channels.

Recommended Tech Stack

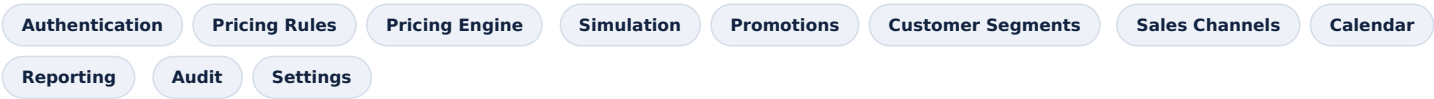
Chosen for reliability and speed of iteration, not novelty.

Layer	Stack	Why
Frontend	React, Next.js, TypeScript, Tailwind CSS, TanStack Query, React Hook Form, Zod	Type-safe, fast to iterate, strong form & data-fetching ecosystem
Backend	NestJS, TypeScript, REST APIs, Swagger, BullMQ	Opinionated module structure that matches the domain-driven design
Database	PostgreSQL	ACID-compliant, strong relational model, handles complex pricing queries well
Cache	Redis	Caches calculated prices, session storage, queue backing
Queue	BullMQ	Scheduled rule publishing, background jobs, reports, notifications
Auth	JWT + Role-Based Access Control	Matches the permission matrix in Document 2
Storage	AWS S3	Reports, CSV imports & exports
Deployment	Docker, Nginx, AWS / Azure	Standard, portable, well-understood ops model

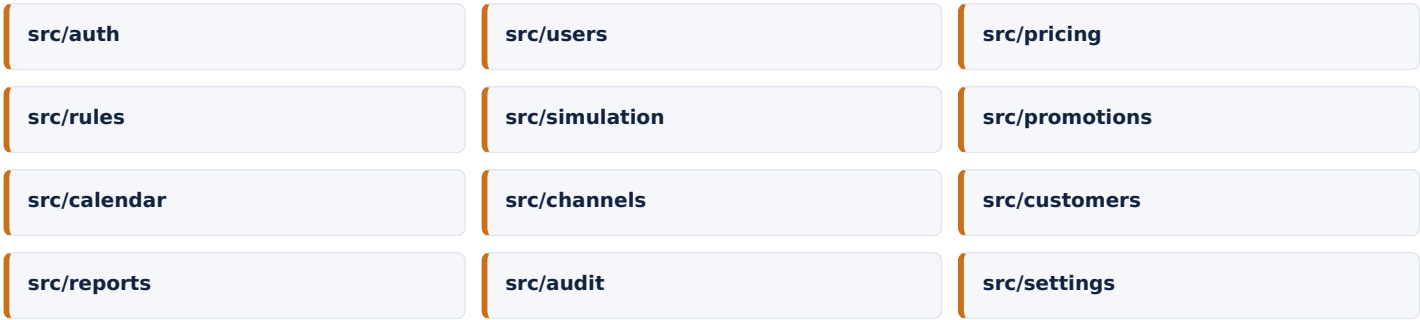
Core Domains & Backend Modules

Folders follow domains, not generic layers.

Each domain owns its own APIs, database tables, business logic, validation, and events:



Backend Module Layout



Two Flows That Matter Most

Creating a rule, and calculating a price — the two paths every request takes.

Request Flow — Creating a Rule



Figure C — From a business user's click to a price a customer actually sees.

Runtime Price Flow

What happens the moment a customer searches.



Figure D — The booking website never calculates a price; it only asks the Pricing Engine for one.

■ **NOTICE**
The website doesn't calculate anything. The **Pricing Engine** does — every channel, every screen, asks the same engine for the same answer.

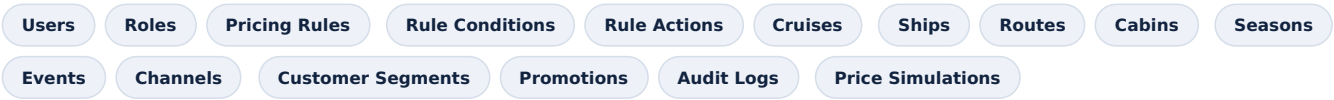
Core Pricing Pipeline

One configurable pipeline computes every final price.



Figure E — Each stage can be reordered, disabled, or extended without touching the others.

Major Database Entities



Non-Functional Requirements

The numbers the architecture is designed to hit.

99.9%

AVAILABILITY

<300ms

PRICE API RESPONSE

1M+

PRICING REQUESTS SUPPORTED

0

PARTIALLY PUBLISHED RULES

Category	Requirement
Scalability	Multiple companies, multiple regions, multiple currencies, millions of pricing requests
Security	JWT, RBAC, audit logs, HTTPS, encryption
Reliability	No pricing rule is ever partially published — publishing is atomic

Future Integrations

Booking Engine

CRM

Payment System

Email

Notification

Analytics

Partner APIs

Deployment Architecture (MVP)

A simple, well-understood path from the internet to storage.

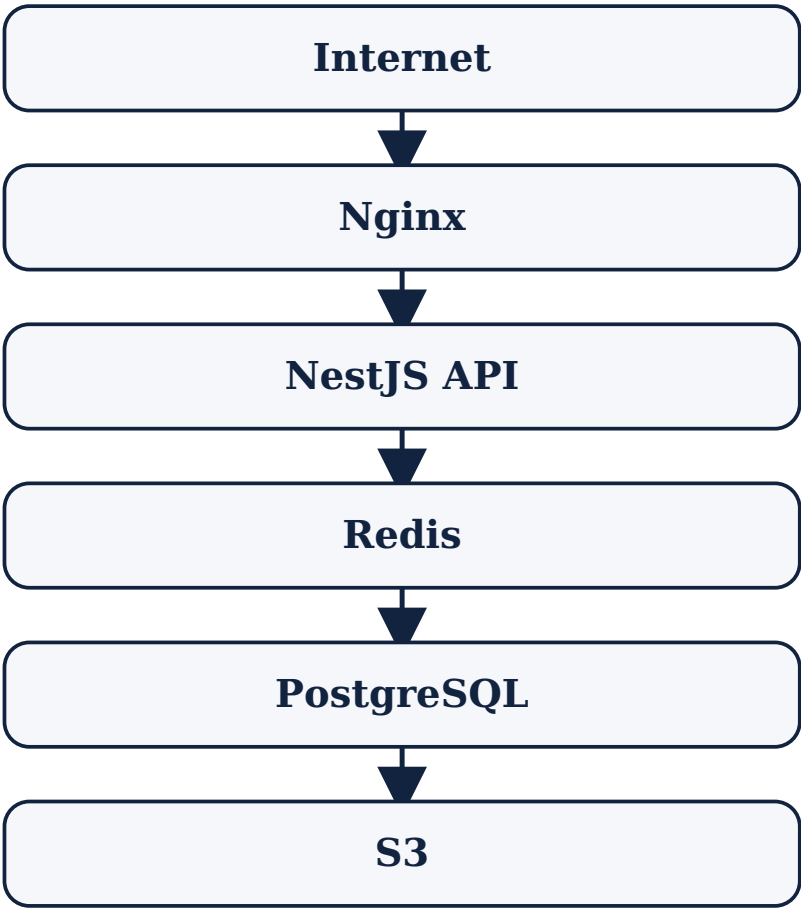
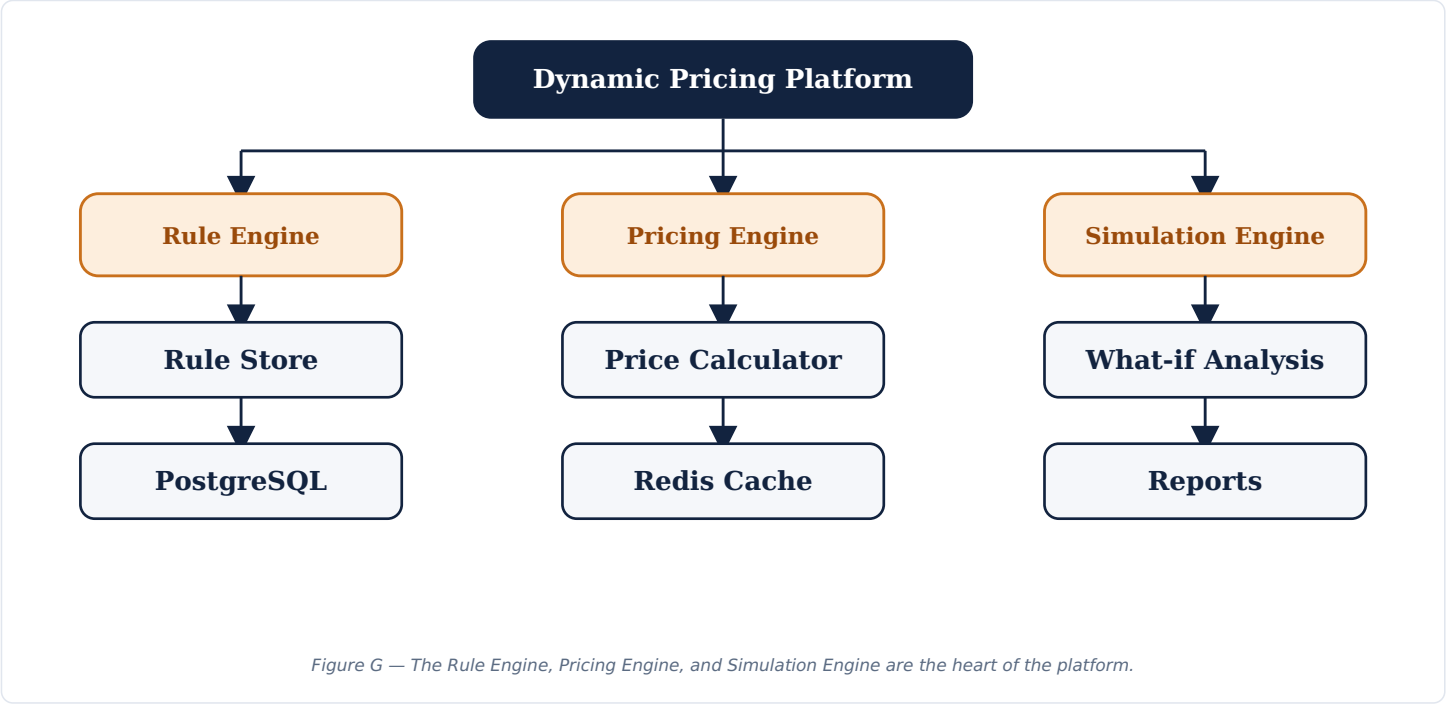


Figure F — Nginx fronts the NestJS API; Redis and PostgreSQL sit behind it; S3 holds reports and exports.

Principal Engineer Review — Three Core Engines

Not CRUD modules. Three engines, and everything else in support of them.

Everything in the architecture so far is technically correct — but it shouldn't be built around CRUD modules. It should be built around three core engines. Users, promotions, reports, and the dashboard are all just supporting functionality around them.



A Rules Engine, Not a Pricing Dashboard

The distinction that changes everything downstream.

 **THE BIGGEST ARCHITECTURAL RECOMMENDATION**

This isn't a "pricing dashboard." It's a **Business Rules Engine for Revenue Management**. That's a fundamentally different architecture — instead of storing prices, the platform stores business knowledge.

A Rule, In Plain Terms

IF	occupancy > 80%
IF	booking within 7 days
IF	channel = Expedia
IF	customer = VIP
THEN	apply pricing action

The most critical component isn't the dashboard — it's the **Rule Engine**, which evaluates configurable business rules and produces the final price. Design that engine correctly, and adding new pricing strategies, AI recommendations, or entirely new travel products later becomes far easier.

 **CURIOUS?**

If pricing is just one type of rule, what else could this engine eventually govern — approval workflows? Refund policies? Loyalty tiers? That's the extensibility this architecture is built for.

Coming Next

Document 4 — the Technical Implementation Plan — translates this architecture into a phased build plan: API contracts, database schema, sprint sequencing, and the rule engine's execution logic in detail.